

COMPONENTE CURRICULAR:	Desenvolvimento de Sistemas II
NOME COMPLETO DO ALUNO:	Murillo Ressimeti Silva
RA:	10732430

Análise Comparativa das Arquiteturas

1. Arquitetura em Camadas (N-Tier / Multicamadas)

A arquitetura em camadas organiza o sistema dividindo as responsabilidades horizontalmente (geralmente em Apresentação/UI, Regras de Negócio/Application e Acesso a Dados/Data Access).

- **Separação de Responsabilidades:** Melhora o cenário atual do monólito bagunçado, pois garante que a interface não mexa direto no banco. No entanto, o sistema **continua sendo um monólito** no deploy. Todas as regras de negócio de todos os domínios (usuários, catálogo, pedidos) ainda rodam juntas.
- **Escalabilidade:** É limitada. Se o módulo de catálogo de produtos sofrer um pico de acessos (por exemplo, na Black Friday), teremos que escalar a aplicação inteira (fazer deploy de várias instâncias do monólito em camadas), o que consome recursos de forma ineficiente.
- **Manutenção e Evolução:** Como o acoplamento físico ainda é alto, alterar uma regra de negócio no módulo de pedidos pode quebrar a emissão de relatórios se as equipes não forem cuidadosas. O deploy continua sendo "tudo ou nada".

2. Arquitetura de Microserviços

Esta abordagem divide o sistema verticalmente por domínios de negócio (Bounded Contexts), onde cada funcionalidade (Cadastro, Catálogo, Pedidos) vira um serviço independente, com seu próprio banco de dados e API.

- **Separação de Responsabilidades:** É máxima. Cada equipe cuida de um serviço específico (Time de Checkout, Time de Catálogo). A dependência entre os times despenca, pois a comunicação entre os serviços é feita por contratos de API claros (REST ou mensageria).

- **Escalabilidade:** É granular e eficiente. Se o processamento de pedidos estiver sobrecarregado, escalamos apenas o microsserviço de pedidos, sem precisar gastar infraestrutura replicando o gerador de relatórios.
- **Manutenção e Evolução:** Permite deploy independente. O time de catálogo pode lançar uma feature nova na quarta-feira sem precisar esperar pelo deploy do sistema inteiro e sem risco de derrubar o módulo de usuários.

Recomendação de Solução

Recomendação: Arquitetura de Microsserviços.

Justificativa:

Considerando o momento atual da **MackCommerce**, a migração para uma Arquitetura em Camadas resolveria apenas a organização interna do código, mas **não resolveria os principais gargalos de negócio da empresa**, que são a escalabilidade e a alta dependência entre as equipes de desenvolvimento.

A escolha por microsserviços se justifica pelos seguintes pilares:

1. **Escalabilidade Direcionada:** E-commerces possuem um comportamento de carga muito oscilante (carrinho e catálogo exigem muito mais do que o cadastro de usuários). Os microsserviços permitem tratar essa demanda de forma isolada e elástica.
2. **Autonomia das Equipes:** Como a empresa quer "lançar novas funcionalidades com agilidade", os microsserviços eliminam o bloqueio entre os times. Cada squad ganha propriedade sobre seu serviço, acelerando o *time-to-market*.
3. **Evolução Tecnológica:** Se no futuro a equipe precisar processar os relatórios usando uma tecnologia mais performática (como Python para dados ou .NET para performance), ela poderá fazer isso no microsserviço específico, sem precisar converter todo o ecossistema da MackCommerce.